JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class LambdaMetafactory

java.lang.Object
  java.lang.invoke.LambdaMetafactory

```
public class LambdaMetafactory
extends Object
```

Methods to facilitate the creation of simple "function objects" that implement one or more interfaces by delegation to a provided `MethodHandle`, possibly after type adaptation and partial evaluation of arguments. These methods are typically used as *bootstrap methods* for `invokedynamic` call sites, to support the *lambda expression* and *method reference expression* features of the Java Programming Language.

Indirect access to the behavior specified by the provided `MethodHandle` proceeds in order through three phases:

- *Linkage* occurs when the methods in this class are invoked. They take as arguments an interface to be implemented (typically a *functional interface*, one with a single abstract method), a name and signature of a method from that interface to be implemented, a method handle describing the desired implementation behavior for that method, and possibly other additional metadata, and produce a `CallSite` whose target can be used to create suitable function objects. Linkage may involve dynamically loading a new class that implements the target interface. The `CallSite` can be considered a "factory" for function objects and so these linkage methods are referred to as "metafactories".
- *Capture* occurs when the `CallSite`'s target is invoked, typically through an `invokedynamic` call site, producing a function object. This may occur many times for a single factory `CallSite`. Capture may involve allocation of a new function object, or may return an existing function object. The behavior `MethodHandle` may have additional parameters beyond those of the specified interface method; these are referred to as *captured parameters*, which must

be provided as arguments to the `CallSite` target, and which may be early-bound to the behavior `MethodHandle`. The number of captured parameters and their types are determined during linkage.
- *Invocation* occurs when an implemented interface method is invoked on a function object. This may occur many times for a single function object. The method referenced by the behavior `MethodHandle` is invoked with the captured arguments and any additional arguments provided on invocation, as if by `MethodHandle.invoke(Object...)`.

It is sometimes useful to restrict the set of inputs or results permitted at invocation. For example, when the generic interface `Predicate<T>` is parameterized as `Predicate<String>`, the input must be a `String`, even though the method to implement allows any `Object`. At linkage time, an additional `MethodType` parameter describes the "instantiated" method type; on invocation, the arguments and eventual result are checked against this `MethodType`.

This class provides two forms of linkage methods: a standard version (`metafactory(MethodHandles.Lookup, String, MethodType, MethodType, MethodHandle, MethodType)`) using an optimized protocol, and an alternate version `altMetafactory(MethodHandles.Lookup, String, MethodType, Object...)`). The alternate version is a generalization of the standard version, providing additional control over the behavior of the generated function objects via flags and additional arguments. The alternate version adds the ability to manage the following attributes of function objects:

- *Bridging.* It is sometimes useful to implement multiple variations of the method signature, involving argument or return type adaptation. This occurs when multiple distinct VM signatures for a method are logically considered to be the same method by the language. The flag `FLAG_BRIDGES` indicates that a list of additional `MethodType`s will be provided, each of which will be implemented by the resulting function object. These methods will share the same name and instantiated type.
- *Multiple interfaces.* If needed, more than one interface can be implemented by the function object. (These additional interfaces are typically marker interfaces with no methods.) The flag `FLAG_MARKERS` indicates that a list of additional interfaces will be provided, each of which should be implemented by the resulting function object.
- *Serializability.* The generated function objects do not generally support serialization. If desired, `FLAG_SERIALIZABLE` can be used to indicate that the function objects should be serializable. Serializable function objects will use, as their serialized form, instances of the class `SerializedLambda`, which requires additional assistance from the capturing class (the class described by the `MethodHandles.Lookup` parameter `caller`); see `SerializedLambda` for details.

Assume the linkage arguments are as follows:

- `invokedType` (describing the `CallSite` signature) has K parameters of types (D1..Dk) and return type Rd;
- `samMethodType` (describing the implemented method type) has N parameters, of types (U1..Un) and return type Ru;

- `implMethod` (the `MethodHandle` providing the implementation has M parameters, of types (A1..Am) and return type Ra (if the method describes an instance method, the method type of this method handle already includes an extra first argument corresponding to the receiver);
- `instantiatedMethodType` (allowing restrictions on invocation) has N parameters, of types (T1..Tn) and return type Rt.

Then the following linkage invariants must hold:

- Rd is an interface
- `implMethod` is a *direct method handle*
- `samMethodType` and `instantiatedMethodType` have the same arity N, and for i=1..N, Ti and Ui are the same type, or Ti and Ui are both reference types and Ti is a subtype of Ui
- Either Rt and Ru are the same type, or both are reference types and Rt is a subtype of Ru
- K + N = M
- For i=1..K, Di = Ai
- For i=1..N, Ti is adaptable to Aj, where j=i+k
- The return type Rt is void, or the return type Ra is not void and is adaptable to Rt

Further, at capture time, if `implMethod` corresponds to an instance method, and there are any capture arguments (`K > 0`), then the first capture argument (corresponding to the receiver) must be non-null.

A type Q is considered adaptable to S as follows:

Q	S	Link-time checks	Invocation-time checks
Primitive	Primitive	Q can be converted to S via a primitive widening conversion	None
Primitive	Reference	S is a supertype of the Wrapper(Q)	Cast from Wrapper(Q) to S
Reference	Primitive	for parameter types: Q is a primitive wrapper and Primitive(Q) can be widened to S for return types: If Q is a primitive wrapper, check that Primitive(Q) can be widened to S	If Q is not a primitive wrapper, cast Q to the base Wrapper(S); for example Number for numeric types
Reference	Reference	for parameter types: S is a supertype of Q for return types: none	Cast from Q to S

API Note:

`These linkage methods are designed to support the evaluation of` *`lambda expressions`* `and` *`method references`* `in the Java Language. For every lambda expressions or method reference in the source code, there is a`

target type which is a functional interface. Evaluating a lambda expression produces an object of its target type. The recommended mechanism for evaluating lambda expressions is to desugar the lambda body to a method, invoke an invokedynamic call site whose static argument list describes the sole method of the functional interface and the desugared implementation method, and returns an object (the lambda object) that implements the target type. (For method references, the implementation method is simply the referenced method; no desugaring is needed.)

The argument list of the implementation method and the argument list of the interface method(s) may differ in several ways. The implementation methods may have additional arguments to accommodate arguments captured by the lambda expression; there may also be differences resulting from permitted adaptations of arguments, such as casting, boxing, unboxing, and primitive widening. (Varargs adaptations are not handled by the metafactories; these are expected to be handled by the caller.)

Invokedynamic call sites have two argument lists: a static argument list and a dynamic argument list. The static argument list is stored in the constant pool; the dynamic argument is pushed on the operand stack at capture time. The bootstrap method has access to the entire static argument list (which in this case, includes information describing the implementation method, the target interface, and the target interface method(s)), as well as a method signature describing the number and static types (but not the values) of the dynamic arguments and the static return type of the invokedynamic site.

Implementation Note:

The implementation method is described with a method handle. In theory, any method handle could be used. Currently supported are direct method handles representing invocation of virtual, interface, constructor and static methods.

Field Summary

Fields

Modifier and Type	Field	Description
static int	**FLAG_BRIDGES**	Flag for alternate metafactories indicating the lambda object requires additional bridge methods
static int	**FLAG_MARKERS**	Flag for alternate metafactories indicating the lambda object implements other marker interfaces besides Serializable

static int	FLAG_SERIALIZABLE	Flag for alternate metafactories indicating the lambda object must be serializable

Constructor Summary

Constructors

Constructor	Description
LambdaMetafactory()	

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static CallSite	altMetafactory(MethodHandles.Lookup caller, String invokedName, MethodType invokedType, Object... args)	Facilitates the creation of simple "function objects" that implement one or more interfaces by delegation to a provided MethodHandle, after appropriate type adaptation and partial evaluation of arguments.
static CallSite	metafactory(MethodHandles.Lookup caller, String invokedName, MethodType invokedType, MethodType samMethodType, MethodHandle implMethod, MethodType instantiatedMethodType)	Facilitates the creation of simple "function objects" that implement one or more interfaces by delegation to

		a provided **MethodHandle**, after appropriate type adaptation and partial evaluation of arguments.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

FLAG_SERIALIZABLE

```
public static final int FLAG_SERIALIZABLE
```

Flag for alternate metafactories indicating the lambda object must be serializable

See Also:

Constant Field Values

FLAG_MARKERS

```
public static final int FLAG_MARKERS
```

Flag for alternate metafactories indicating the lambda object implements other marker interfaces besides Serializable

See Also:

Constant Field Values

FLAG_BRIDGES

```
public static final int FLAG_BRIDGES
```

Flag for alternate metafactories indicating the lambda object requires additional bridge methods

See Also:

Constant Field Values

Constructor Detail

LambdaMetafactory

```
public LambdaMetafactory()
```

Method Detail

metafactory

```
public static CallSite metafactory(MethodHandles.Lookup caller,
                                   String invokedName,
                                   MethodType invokedType,
                                   MethodType samMethodType,
                                   MethodHandle implMethod,
                                   MethodType instantiatedMethodType)
                            throws LambdaConversionException
```

Facilitates the creation of simple "function objects" that implement one or more interfaces by delegation to a provided `MethodHandle`, after appropriate type adaptation and partial evaluation of arguments. Typically used as a *bootstrap method* for `invokedynamic` call sites, to support the *lambda expression* and *method reference expression* features of the Java Programming Language.

This is the standard, streamlined metafactory; additional flexibility is provided by `altMetafactory(MethodHandles.Lookup, String, MethodType, Object...)`. A general description of the behavior of this method is provided `above`.

When the target of the `CallSite` returned from this method is invoked, the resulting function objects are instances of a class which implements the interface named by the return type of `invokedType`, declares a method with the name given by `invokedName` and the signature given by `samMethodType`. It may also override additional methods from `Object`.

Parameters:

`caller - Represents a lookup context with the accessibility privileges of the caller. When used with invokedynamic, this is stacked automatically by the VM.`

invokedName - The name of the method to implement. When used with invokedynamic, this is provided by the NameAndType of the InvokeDynamic structure and is stacked automatically by the VM.

invokedType - The expected signature of the CallSite. The parameter types represent the types of capture variables; the return type is the interface to implement. When used with invokedynamic, this is provided by the NameAndType of the InvokeDynamic structure and is stacked automatically by the VM. In the event that the implementation method is an instance method and this signature has any parameters, the first parameter in the invocation signature must correspond to the receiver.

samMethodType - Signature and return type of method to be implemented by the function object.

implMethod - A direct method handle describing the implementation method which should be called (with suitable adaptation of argument types, return types, and with captured arguments prepended to the invocation arguments) at invocation time.

instantiatedMethodType - The signature and return type that should be enforced dynamically at invocation time. This may be the same as samMethodType, or may be a specialization of it.

Returns:

a CallSite whose target can be used to perform capture, generating instances of the interface named by invokedType

Throws:

LambdaConversionException - If any of the linkage invariants described above are violated

altMetafactory

```
public static CallSite altMetafactory(MethodHandles.Lookup caller,
                                      String invokedName,
                                      MethodType invokedType,
                                      Object... args)
                               throws LambdaConversionException
```

Facilitates the creation of simple "function objects" that implement one or more interfaces by delegation to a provided MethodHandle, after appropriate type adaptation and partial evaluation of arguments. Typically used as a *bootstrap method* for invokedynamic call sites, to support the *lambda expression* and *method reference expression* features of the Java Programming Language.

This is the general, more flexible metafactory; a streamlined version is provided by metafactory(java.lang.invoke.MethodHandles.Lookup,

String, MethodType, MethodType, MethodHandle, MethodType). A general description of the behavior of this method is provided above.

The argument list for this method includes three fixed parameters, corresponding to the parameters automatically stacked by the VM for the bootstrap method in an invokedynamic invocation, and an Object[] parameter that contains additional parameters. The declared argument list for this method is:

```
CallSite altMetafactory(MethodHandles.Lookup caller,
                        String invokedName,
                        MethodType invokedType,
                        Object... args)
```

but it behaves as if the argument list is as follows:

```
CallSite altMetafactory(MethodHandles.Lookup caller,
                        String invokedName,
                        MethodType invokedType,
                        MethodType samMethodType,
                        MethodHandle implMethod,
                        MethodType instantiatedMethodType,
                        int flags,
                        int markerInterfaceCount,  // IF flags has MARKERS set
                        Class... markerInterfaces, // IF flags has MARKERS set
                        int bridgeCount,           // IF flags has BRIDGES set
                        MethodType... bridges      // IF flags has BRIDGES set
                        )
```

Arguments that appear in the argument list for metafactory(MethodHandles.Lookup, String, MethodType, MethodType, MethodHandle, MethodType) have the same specification as in that method. The additional arguments are interpreted as follows:

- flags indicates additional options; this is a bitwise OR of desired flags. Defined flags are FLAG_BRIDGES, FLAG_MARKERS, and FLAG_SERIALIZABLE.
- markerInterfaceCount is the number of additional interfaces the function object should implement, and is present if and only if the FLAG_MARKERS flag is set.
- markerInterfaces is a variable-length list of additional interfaces to implement, whose length equals markerInterfaceCount, and is present if and only if the FLAG_MARKERS flag is set.
- bridgeCount is the number of additional method signatures the function object should implement, and is present if and only if the FLAG_BRIDGES flag is set.

- `bridges` is a variable-length list of additional methods signatures to implement, whose length equals `bridgeCount`, and is present if and only if the `FLAG_BRIDGES` flag is set.

Each class named by `markerInterfaces` is subject to the same restrictions as `Rd`, the return type of `invokedType`, as described `above`. Each `MethodType` named by `bridges` is subject to the same restrictions as `samMethodType`, as described `above`.

When FLAG_SERIALIZABLE is set in `flags`, the function objects will implement `Serializable`, and will have a `writeReplace` method that returns an appropriate `SerializedLambda`. The `caller` class must have an appropriate `$deserializeLambda$` method, as described in `SerializedLambda`.

When the target of the `CallSite` returned from this method is invoked, the resulting function objects are instances of a class with the following properties:

- The class implements the interface named by the return type of `invokedType` and any interfaces named by `markerInterfaces`
- The class declares methods with the name given by `invokedName`, and the signature given by `samMethodType` and additional signatures given by `bridges`
- The class may override methods from `Object`, and may implement methods related to serialization.

Parameters:

`caller - Represents a lookup context with the accessibility privileges of the caller. When used with invokedynamic, this is stacked automatically by the VM.`

`invokedName - The name of the method to implement. When used with invokedynamic, this is provided by the NameAndType of the InvokeDynamic structure and is stacked automatically by the VM.`

`invokedType - The expected signature of the CallSite. The parameter types represent the types of capture variables; the return type is the interface to implement. When used with invokedynamic, this is provided by the NameAndType of the InvokeDynamic structure and is stacked automatically by the VM. In the event that the implementation method is an instance method and this signature has any parameters, the first parameter in the invocation signature must correspond to the receiver.`

`args - An Object[] array containing the required arguments samMethodType, implMethod, instantiatedMethodType, flags, and any optional arguments, as described altMetafactory(MethodHandles.Lookup, String, MethodType, Object...) above}`

Returns:

a CallSite whose target can be used to perform capture, generating instances of the interface named by invokedType

Throws:

`LambdaConversionException` - If any of the linkage invariants described above are violated

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.